

MENTOR-RL Methods

Peter Kruse

June 2026

0.1 Introduction and Motivation

0.1.1 Biological Interpretation as Mechanistic Discovery

Biological interpretation involves the development and discovery of mechanistic relationships between biological entities such as genes, proteins, RNA, and other molecular components. Systems biology approaches this by studying the interactions between molecular entities rather than their individual properties [17, 5], seeking to identify *modules*, groups of functionally related entities, and to characterize the mechanisms underlying their behavior. Because these interactions span multiple scales and regulatory layers, modern analysis relies on multiplex and multilayer representations that hold the same genes across several types of edge [4, 19, 28, 14], over which propagation methods such as random walk with restart (RWR) identify functionally related neighborhoods [22, 13]. Module-level analysis has become foundational for identifying disease mechanisms, prioritizing drug targets, and interpreting polygenic phenotypes [27]. Machine learning and artificial intelligence techniques such as iterative random forest (iRF) [6, 11] have accelerated this work by constructing biologically informed networks that scientists explore to generate mechanistic and functional hypotheses.

As these methods increase the resolution and scope of networks, biological interpretation has become increasingly mediated by large, multi-scale graph structures. To comprehensively examine these large biological networks and supporting databases, mechanistic exploration requires iterative querying and multi-step reasoning. Recent agentic interpretation systems such as eubiota [25] and MENTOR-IA [47] demonstrate that provenance-preserving retrieval and tool-mediated synthesis can yield high-fidelity mechanistic narratives. However, these pipelines depend on frontier model inference and are not fully optimized as a trainable policy.

0.1.2 Biological Interpretation is Bottlenecked by Human Cognition and Frontier Model Cost

Although ML/AI methods such as MENTOR [44] have refined mechanistic focus into groups of functionally related modules of genes, these modules can still be massive and intractable for individual researchers to systematically explore. Additionally, most research emphasizes a deep-but-narrow focus on a small subset of functionally related genes, which can overlook broader, yet biologically meaningful, network context, particularly in highly interactive systems such as living organisms. As biological networks grow in complexity, the combinatorial space of possible multi-hop interactions expands rapidly, while human interpretive capacity remains fixed.

While large language models (LLMs) can synthesize large volumes of information, frontier inference cost scales with exploration length and reasoning depth, limiting agentic deployment for long-horizon network traversal. This mismatch creates a scaling limit in biological mechanistic interpretation: rich relational structures can be

computed, but downstream synthesis remains constrained by human bandwidth and model cost. Consequently, the speed, breadth, and contextual depth of mechanistic discovery are limited, particularly for complex traits, polygenic diseases, and multi-layer regulatory systems.

0.1.3 AI-Guided Reasoning to Extend Mechanistic Interpretation Beyond Human Scale

Open-source LLMs present an opportunity to aid scientists in mechanistic discovery tasks without being cost prohibitive. Language models reason more reliably when prompted to produce intermediate steps [49, 23], and equipping them with external tools supplies a layer of deterministic verification that reduces hallucination and preserves provenance [39, 52]. Adding an explicit verification stage improves factual reliability further, whether through iterative self-critique without weight updates [26, 42] or through a dedicated verifier with separate weights [25]. Applied to gene set interpretation, this progression runs from precomputed language-model embeddings [9], through multi-step pipelines with database-backed self-verification [48], to multi-agent mechanistic synthesis [47, 25]. None of these systems is trained end-to-end as a policy over verifiable rewards.

Reinforcement learning supplies the machinery to close that gap. Preference optimization aligns a policy to pairwise judgments, and direct preference optimization (DPO) does so without a separate learned reward model [10, 33, 36]. Policy gradient methods instead optimize against scalar rewards over complete rollouts [40, 41], and verifiable rewards have been shown to elicit long-horizon reasoning behavior at frontier scale [16]. These methods have been applied to tool-using agents for browsing, large API vocabularies, and multi-step web tasks [30, 35, 34], but not to a policy whose rewards are grounded in network-derived biological targets. We propose MENTOR-RL, a model trained on biological ground truth and verifiable interaction objectives, enabling scalable mechanistic exploration without reliance on proprietary models. Leveraging open-source model offerings allows local deployment, with training costs amortized across many exploration sessions. Rather than replacing scientific judgment, MENTOR-RL extends the interpretive bandwidth of researchers, enabling broader, faster, and more context-aware mechanistic discovery.

0.1.4 A World Model of the Multiplex, Not a Graph Lookup Table

Sequence models trained on synthetic tasks form internal, causal models of the process behind their data, and those models generalize to states never seen during training [24]. This works because the rules that generate the data are compact and recur across many examples, while the specific data does not. Two consequences shape this proposal. First, a model can learn the operators that regenerate structure,

but it cannot memorize irreducible content that follows no compact rule, so biological edge weights, walk scores, and distances belong in the runtime rather than in the weights. Second, a world model emerges only under dense sampling of the generating process, which a single network queried with fixed templates cannot supply.

We therefore train MENTOR-RL on a structured population of multiplex networks rather than on one. Every network in the population is built by the same rule: the predictive expression network (PEN) layers selected for a biological context define its gene universe, the remaining layers are induced on that universe, and the layers are stacked. Varying the context, whether a cell type, a tissue, a brain region, or an organ system, varies the resulting network while holding the construction rule fixed. We call this population the *multiplex lattice*, and it supplies the dense sampling that a single network cannot.

The invariant we train on is therefore not any particular network or hierarchy, but the operator that maps a multiplex to its structure and the way layer composition drives that operator. We expect hierarchies to differ strongly across contexts, with distinct clades emerging in different cell types and regions. That divergence is the biology rather than an artifact: a model that has learned the operator will produce correspondingly different hierarchies for different contexts, and recovering the divergence correctly is a stronger result than reproducing conservation.

A hierarchy alone cannot serve as the world model, because it is a lossy abstraction of the multiplex it summarizes. Two clades that sit far apart in a tree may still be joined by short, biologically supported paths through bridge nodes in the underlying network, which is why a signal such as drug-target enrichment can appear in several distant branches and yet act through a single connected system. The model must hold the network itself and reason about the cross-clade connectivity that the tree suppresses. The hierarchy is the index used to find and prioritize; the network is where hypotheses are confirmed.

This proposal pursues four objectives. We define the multiplex lattice and ground it in a standard biological ontology, so that every network carries a canonical address and network composition is the join operation over that ontology. We design a supervised curriculum that teaches multiplex operators rather than graph content, holding the three structural views as co-equal targets and admitting a training example only when its target passes a typed validator against deterministic runtime output. We hand the resulting checkpoint to a tool-using agent, formalized as a sequential decision process over a structured state and scored by deterministic, schema-grounded rewards rather than proprietary judges. Finally, we make the world-model claim falsifiable through two flagship evaluations, generating a hierarchy for a context never seen in training and projecting a drug target onto the hierarchy before returning to the network to recover the connectivity that justifies it, supported by activation probes that test whether the recovered structure is causal rather than decorative.

0.2 Proposed Methodology

Our methodology has four parts. Sections 0.2.1 through 0.2.3 define the training substrate: the lattice of multiplex networks, the ontology that addresses it, and the three structural views the model is required to hold. Sections 0.2.6 and ?? describe the deterministic runtime and the supervised curriculum that builds the world model, and Section ?? defines the split and leakage protocol. Sections 0.2.4 through 0.2.8 formalize the mechanistic agent that the foundation checkpoint hands off to, together with its task corpora, trajectory generation, and reward functions. Section 0.2.9 defines the staged training pipeline and curriculum, and Section 0.2.11 describes evaluation and benchmarking.

0.2.1 The Multiplex Lattice

Every multiplex in the lattice is built by one rule. The PEN layers selected for a context define its gene universe; HumanNet and the other non-PEN layers are induced on that universe, restricted to those genes and the edges among them; and all layers are then stacked into a multiplex. The backbone is therefore not shared across the lattice. It is an induced subgraph whose nodes and edges both follow from the PEN selection, so two contexts differ in their expression layers and in their induced backbone together.

The family this rule generates is nested and compositional. The finest grain is a single cell type, in one tissue, in one region. Combining cell types gives a tissue-level multiplex within a region, combining tissues gives a region-level multiplex, and combining regions builds up to the whole-brain multiplex we already hold. The same construction extends to other organ systems and to multiplexes that join organ systems, up to a whole-body multiplex. Disease-relevant networks, including the anxiety, suicide, pan-substance-use-disorder, and cardiac projections, are points in this same lattice rather than separate resources, which is what makes transfer between them meaningful.

Composition applies the same rule to a union. The gene universe of a combined multiplex is the union of its parts' PEN gene sets, the non-PEN layers are re-induced on that union, and the PEN layers are stacked. This has a direct consequence for training: a change of context is compound. Adding or removing a single PEN layer changes the gene universe and re-induces the backbone along with it, so a minimal pair of multiplexes differs by that layer together with its induced backbone, and the isolated single-layer effect is recovered only under a controlled ablation.

Before the full build, a divergence validation study confirms that different PEN selections do in fact produce divergent multiplexes, hierarchies, and connectivity, by comparing RWR-LOE neighborhoods, MENTOR-derived hierarchies, and cross-clade connectivity across a first set of contexts. Strong divergence is the expected and desired result, and it is a precondition for everything downstream: it is what establishes that the lattice samples a generating process rather than re-presenting a

single network under different names. This study also fixes parameters that remain provisional until it completes, including the sampling rates across lattice tiers and how many contexts are worth building at each tier.

0.2.2 Context Ontology and Lattice Addressing

The lattice is represented as a grounded ontology, so that every multiplex has a canonical and interoperable address and the composition operator has a defined domain. We build on existing community ontologies rather than a bespoke scheme, which lets the lattice inherit versioning, cross-species bridges, and alignment to the public single-cell corpora that also serve as data sources.

Each context is placed on two axes, with an optional third. Anatomy and regions are grounded in UBERON [29], extended by human brain atlas terms where finer regional detail is required than UBERON carries. Cell types are grounded in the Cell Ontology [15], extended by the Provisional Cell Ontology for the transcriptomic brain types the Cell Ontology does not yet hold [45]. Developmental stage, where it matters, is grounded in HsapDv. The linkage between the two axes comes from the ASCT+B tables of the Human Reference Atlas [8], which tie anatomical structures to their resident cell types and are maintained across the whole body, and node annotation follows the CELLxGENE schema [1] so that contexts align with public single-cell corpora. Because the Human Reference Atlas is a whole-body construction, extending the lattice from brain to other organ systems populates more of the same ontologies rather than replacing the stack.

A context is a node addressed by an anatomy term, a cell-type term, and a developmental stage, and it carries the PEN layers that define its gene universe together with the taxonomy cluster identifier that produced them. The resulting structure is a partial order rather than a tree, because a cell type recurs across regions and a combination joins several parents: a single cell type in a region sits beneath both the all-cell-types node for that region and the same cell type across the parent structure. Composition is the join in that partial order. It moves anatomy up the part-of hierarchy and cell types up the transcriptomic taxonomy from subcluster to cluster to supercluster, and the joined node’s gene universe is the union of its parts’ PEN gene sets with the non-PEN layers re-induced on it, exactly as Section 0.2.1 specifies. Each node stores both identities, the ontology term for interoperability and the cluster identifier for provenance, and both are version-pinned alongside the graph version and layer-list hash, with an explicit mapping from PEN-derived types to taxonomy identifiers that keeps node identity stable when a taxonomy is re-versioned.

The ontology is supplied to the model as context and serves as the domain of the composition operator; it is not content baked into the weights. The model reads a provided ontology and navigates it, so the brain ontology is context and the same navigation transfers when the lattice extends to other organ systems. Navigation is trained as a relational operator: returning a node’s parents, children, and siblings, naming the join of two nodes as the multiplex their union defines, and reporting

the ontology distance between two contexts. That distance also indexes evaluation, letting us ask how strongly sibling cell types diverge relative to how strongly a child diverges from its parent.

0.2.3 Three Structural Views

The model is trained to hold three views of a multiplex, weighted equally, because each captures structure the others suppress.

The bottom-up view is the gene-centric neighborhood produced by random walk with restart lines of evidence (RWR-LOE) [21]. Applied to a seed gene, RWR-LOE walks the entire multiplex, treating each layer as an independent line of evidence, and returns every other gene ranked by propagation proximity to that seed. Rather than imposing a fixed neighborhood size, a geometric elbow filter places the cutoff at the bend in the score-versus-rank curve, so retaining the seed together with all genes above the cutoff yields one adaptive, seed-centered module per gene in the network. Operators over this view include rank and score lookup, comparison of two genes within one ranked vector, closest- k retrieval, elbow reasoning, set relations between neighborhoods, leave-one-out support, layer ablation, and seed essentiality.

The top-down view is the hierarchy produced by MENTOR (Multiplex Embedding of Networks for Team-Based Omics Research) [44], which computes RWR embeddings for every gene across the layers of a multiplex, converts those embeddings into pairwise distances, and applies hierarchical clustering to yield a genome-wide dendrogram whose leaves are genes and whose internal nodes represent successively coarser functional groups. Cutting the dendrogram at a given merge height selects a family of nested modules spanning the entire gene universe of the network, so granularity is controlled by the height of the cut. Operators over this view include clade membership, the lowest common ancestor of a gene pair, the subtree induced by a cut, parent, child, and sibling relations, and within-clade cohesion. Every one of these targets is defined inside a single tree and is therefore alignment-free: hierarchies are compared across contexts by value, and no two trees ever need to be aligned to each other. This is what makes divergence trainable rather than an obstacle. Cut height doubles as a difficulty control, running from coarse, well-separated clades to fine, easily confused ones.

The third view is the connectivity of the underlying network, which the hierarchy necessarily suppresses. Because a dendrogram is a summary of a multiplex, distance in the tree is not distance in the network: two clades on distant branches may be joined by short, layer-supported paths through a small number of bridge nodes. Operators over this view include cross-clade path recovery, connector and bridge node identification, decomposition of a path by its supporting layers, cohesion tested against a null distribution, and calibrated refusal of the inference that separation in the tree implies separation in the network. Training this view as a first-class target is what keeps the model from reading the dendrogram as the whole truth, and it

carries the calibration rules that prevent claiming a direct interaction where only a cross-layer route exists.

The three views are exercised together in the projection operation of Section ??, which seeds a neighborhood on the network, tests that neighborhood against the hierarchy, and then returns to the network to justify what the hierarchy reported.

0.2.4 Agent Loop Formulation

Loop Formalization

We model mechanistic graph exploration as a sequential decision process executed by a single policy model with an internal self-verification step. The loop begins when the user provides a query, q , and optional evidence e^{user} , which may include text context, seed genes, or a multiplex network. These inputs initialize the working interpretation I_0 and the structured state S_0 .

For each decision point $t < T_{\max}$, the current decision context is

$$D_t = (q, e^{\text{user}}, I_t, S_t).$$

Given D_t , the agent emits an action consisting of a textual reasoning step r_t and an optional tool action a_t . The reasoning step specifies the current subgoal or interpretation update, while the tool action specifies a runtime-executable call. If a tool action is emitted, it is executed by a deterministic runtime ε ; otherwise, the observation is empty.

After the action is produced and any deterministic tool calls are executed, the same policy enters a verification mode. Conditioned on the query, user-provided evidence, the current interpretation, the current structured state, and the current reasoning/action outputs, the verifier updates the interpretation and structured state, yielding (I_{t+1}, S_{t+1}) . It also emits a continuation decision,

$$z_t \in \{\text{continue}, \text{revise}, \text{stop}\},$$

which determines whether the current interpretation should be extended, revised, or terminated. The decision **revise** indicates that the next iteration proceeds by correcting the interpretation and structured state rather than simply extending them.

The process repeats until either $z_t = \text{stop}$ or the maximum decision budget $T_{\max}$ is reached. At termination, I_T and S_T are returned as outputs. I_T provides a human-readable answer to the query with respect to the accumulated evidence, while S_T stores the corresponding verifiable outputs for reward computation and provenance preservation.

We formalize one iteration of the agent loop as

$$r_t, a_t \sim \pi_{\theta}^{\text{act}}(\cdot \mid D_t), \quad o_t = \varepsilon(a_t), \quad I_{t+1}, S_{t+1}, z_t \sim \pi_{\theta}^{\text{verify}}(\cdot \mid r_t, a_t, o_t, D_t),$$

where r_t is the agent’s textual reasoning or subgoal, a_t is the optional tool action, and o_t is the observation returned by the deterministic runtime ε , with $\varepsilon(\emptyset) = \emptyset$

Algorithm 1 The MENTOR-RL agent loop

Require: query q , user evidence e^{user} , maximum decision budget T_{max} , deterministic runtime ε

```
1: Initialize working interpretation  $I_0$  and structured state  $S_0$  from  $(q, e^{\text{user}})$ 
2:  $T \leftarrow 0$ 
3: for  $t = 0, 1, \dots, T_{\text{max}} - 1$  do
4:    $D_t \leftarrow (q, e^{\text{user}}, I_t, S_t)$ 
5:    $(r_t, a_t) \sim \pi_{\theta}^{\text{act}}(\cdot \mid D_t)$ 
6:   if  $a_t \neq \emptyset$  then
7:      $o_t \leftarrow \varepsilon(a_t)$ 
8:   else
9:      $o_t \leftarrow \emptyset$ 
10:  end if
11:   $(I_{t+1}, S_{t+1}, z_t) \sim \pi_{\theta}^{\text{verify}}(\cdot \mid r_t, a_t, o_t, D_t)$ 
12:   $T \leftarrow t + 1$ 
13:  if  $z_t = \text{stop}$  then
14:    break
15:  end if
16: end for
17: return final interpretation  $I_T$  and structured state  $S_T$ 
```

by convention. The policies $\pi_{\theta}^{\text{act}}$ and $\pi_{\theta}^{\text{verify}}$ denote two prompting modes of the same underlying model, sharing weights so that biological vocabulary and tool-use competence learned in one mode transfers to the other. We treat the risk that the verifier collapses as a monitoring concern during training, discussed in Section 0.4.2. At each decision point, the pair (I_t, S_t) represents the current mechanistic hypothesis, with I_t serving as a human-readable view and S_t as a machine-readable view of the same underlying state. The full agent loop algorithm is detailed in Algorithm 1.

Working Interpretation

The working interpretation I_t is the human-readable form of the current mechanistic hypothesis and contains:

1. the current mechanistic claim,
2. the main evidence supporting that claim,
3. any uncertainty, conflict, or alternative explanation,
4. the next question or subgoal the model is trying to resolve.

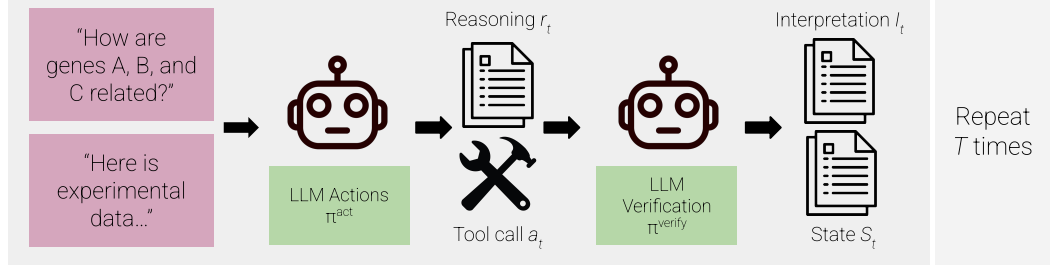


Figure 1: The MENTOR-RL agent loop.

Structured State

The structured state S_t stores the same hypothesis in a machine-readable form suitable for tool execution, reward computation, and provenance tracking. It contains:

1. the user-provided anchors (q, e^{user}) ,
2. the current relationship status (unknown, partially observed group, validated group, multiple groups, or insufficient support) (rel_t) ,
3. the current predicted group or groups (G_t) ,
4. the evidence and provenance collected so far (E_t) ,
5. the current predicted mechanistic labels, each tied to the evidence handles that support it,
6. the tool-use counters recording total and invalid tool calls (n_{total}, n_{inv}) ,

7. the remaining budget and continuation state, and, at termination, the recorded termination signal $((T_{\max} - T), z_t)$.

0.2.5 Task Outputs and Mechanistic Targets

Final Interpretation

The final outputs of the model are the terminal interpretation I_T and the terminal structured state S_T . Together, these outputs must support four cases: completion of a partially observed mechanistic group, explanation of an already coherent group, refinement of a noisy group, and recognition that no single shared mechanism is supported. The final interpretation states which case is supported by the evidence, summarizes the main evidence for that claim, and links the evidence to the claim through reasoning traces and tool calls. It also records uncertainty or abstention when evidence is incomplete. If the loop terminates with $z_t = \text{stop}$, the next subgoal field is left blank. If the loop terminates because the decision budget $T_{\max}$ is exhausted, the interpretation may include a recommended next subgoal to guide further exploration.

Final Structured State

The final structured state is a schema-enforced JSON object. It contains the same components defined in Section 0.2.4, but with finalized values for relationship status, predicted groups, collected evidence, and mechanistic labels. These values are used for reward evaluation and provenance tracking. The final structured state also records whether termination occurred because the model emitted $z_t = \text{stop}$ or because the budget $T_{\max}$ was exhausted. If the budget is exhausted, the remaining budget is 0; if the model stops early, the remaining budget is $T_{\max} - T$.

Supported Task Cases

We support four output cases during training to reflect the variability of experimental evidence that users may provide:

1. partial group completion,
2. mechanism explanation for an already related group,
3. group refinement from a noisy module,
4. no single shared mechanism.

Partial group completion occurs when the user provides a subset of related genes; the model expands this subset into a larger mechanistically related group and explains it. Mechanism explanation occurs when the user provides a complete module; the model explains the mechanism without unnecessary expansion. Group refinement occurs when the user provides a known module with several unrelated noise genes;

the agent refines the group so that it contains only mechanistically related genes. Finally, in the no shared mechanism case, the user provides genes that do not support one coherent mechanism, and the model either splits them into multiple groups or returns insufficient support.

Targets for Training and Evaluation

We define two complementary target types for training and evaluation. **Module membership targets** are the hidden gene sets drawn from either MENTOR-derived dendrogram modules or elbow-filtered RWR-LOE modules (Sections 0.2.6 and 0.2.6). These targets are only used for scoring: the agent never observes hidden module members and must recover them through exploration. These modules supply the supervised signal for set-level recovery, refinement, and abstention.

Because dendrogram modules are derived computationally from multiplex structure, they offer genome-wide coverage, but often without per-entry experimental annotations. **Mechanistic evidence targets** therefore score the quality of a final explanation against the evidence retrieved during exploration, such as enrichment, annotation, and graph support. This allows interpretations to be rewarded for being grounded in evidence rather than simply matching a pre-defined annotation, which encourages the exploration of novel biology beyond existing databases.

0.2.6 Data, Resources, and Environment

Our dataset is constructed from a combination of existing internal tool infrastructure, open-source biological resources, and graph exploration environments.

Warm-Start SFT Dataset

We first construct a warm-start SFT subset from the MENTOR-IA tool stack [47], including gene metadata, gene ontology (GO) annotations, pathway membership, and literature evidence. These data are retrieved via MyGene.info [51] and Europe PMC [12]. This warm-start subset covers four task families:

1. entity normalization, such as resolving gene names and synonyms to canonical identifiers,
2. annotation retrieval, such as returning GO terms, localization, and pathway information for a gene or gene set,
3. literature-grounded extraction, such as identifying relevant genes, pathways, or mechanisms from retrieved abstracts, and
4. evidence-to-finding generation, which converts retrieved tool outputs into short, grounded mechanistic statements.

We include tool-enabled (open-book) and tool-disabled (closed-book) questions, with open-book examples emphasized to promote schema-valid tool use and evidence grounding, and closed-book examples used to strengthen biological vocabulary and lightweight recall. To make this stage reproducible, API responses and database versions will be cached during dataset construction.

To make the warm-start SFT stage concrete, we define a core tool vocabulary derived from the existing MENTOR-IA infrastructure [47] in Table 1. Early SFT emphasizes schema-valid biological annotation and enrichment calls, while graph-exploration and RWR++ operators are introduced during DPO and online RL.

MENTOR-Derived Training Corpus

After constructing the warm-start SFT dataset, we create two complementary module corpora that together supply the hidden targets for trajectory generation: a MENTOR-derived corpus (this section) and an RWR-LOE corpus (Section 0.2.6). We first parse the genome-wide MENTOR dendrogram into modules, where each module is the gene set of a dendrogram subtree. We partition the modules into train, validation, and test splits to prevent leakage across stages.

For each module C_{module} of size n , we deterministically materialize up to four task instances, one per case in Section 0.2.5. This ensures that all four task types are represented for as many modules as possible rather than sampled independently. The **explanation** instance uses the full module as input, $C^{\text{in}} = C_{\text{module}}$. The **recovery** instance drops k_{drop} genes from the module, $C^{\text{in}} = C_{\text{module}} \setminus G_{\text{drop}}$, and the **refinement** instance adds k_{add} noise genes, $C^{\text{in}} = C_{\text{module}} \cup G_{\text{add}}$. In both the recovery and refinement cases, the hidden target remains $C = C_{\text{module}}$. The number of genes added or dropped scales with the task difficulty and module size, so that harder instances perturb a larger fraction of the module. The **none** instance constructs a pseudo-module of unrelated genes as input with hidden target $C = \emptyset$.

Noise and **none** genes are drawn from difficulty-modulated *dendrogram distance bands*, which are defined by walking up the module’s ancestry in the tree, so that difficulty controls how topologically close the distractors are to the module. Additionally, **none** genes are constrained to be conflict-free (i.e. they are not related to each other), so that sampled genes do not form a coherent module themselves. All selection is deterministic given a fixed seed, making the corpus reproducible. The resulting MENTOR-based corpus provides supervision for schema-valid tool use and for the terminal outputs of Section 0.2.5.

RWR-LOE Module Corpus

In addition to the top-down, hierarchical MENTOR-derived module corpus, we create a corpus of RWR-LOE exploration across all genes in the multiplex to provide a bottom-up complement. For each gene g in the same multiplex universe as the MENTOR dendrogram, we run `rwr_loe( $g$ )` to obtain a propagation score and rank for every

Table 1: Tool vocabulary for the MENTOR-RL agent. Related RWR++ calls are grouped by function while preserving the model-facing tool names.

Tool(s)	Purpose	Primary inputs	Stage
<code>query_mygene</code>	Gene metadata, GO terms, pathways, and summaries	<code>query</code> ; requested <code>fields</code>	SFT
<code>enrich_gene_set</code>	Functional enrichment over a candidate gene set	<code>genes</code> ; <code>background</code> ; <code>top_k</code> ; <code>threshold</code>	SFT, DPO, RL
<code>get_neighbors</code> ; <code>induce_subgraph</code>	Local native graph and induced-subgraph evidence	gene or gene-set identifiers; <code>layers</code>	DPO, RL
<code>rwr</code> ; <code>rwr_loe</code> ; <code>shortest_paths</code>	RWR++ expansion, leave-one-out relatedness, and path evidence	<code>seed</code> , <code>query</code> , <code>source</code> , or <code>target</code> genes; <code>layers</code> ; <code>top_k</code>	DPO, RL
<code>get_rank</code> ; <code>get_distance</code> ; <code>get_spearman</code> ; <code>get_pearson</code> ; <code>get_dot_similarity</code>	Rank, distance, and vector-similarity comparisons	paired genes; <code>metric</code> ; <code>layers</code>	DPO, RL
<code>get_rank_vector_summary</code> ; <code>get_encoding_summary</code>	Compact rank-vector and encoding summaries	<code>seed</code> or <code>query</code> genes; <code>layers</code> ; <code>top_k</code>	DPO, RL
<code>get_gene_layers</code> ; <code>get_nodes_by_layer</code> ; <code>get_layer_stats</code> ; <code>get_path_layer_counts</code> ; <code>get_component_summary</code>	Layer membership, layer statistics, path support, and components	genes or layers, depending on <code>query</code>	DPO, RL
<code>get_seed_essentiality</code> ; <code>get_layer_ablation</code> ; <code>get_node_perturbation</code>	Stability, ablation, and perturbation analyses	<code>seed</code> or <code>perturbation</code> genes; <code>layers</code> ; <code>top_k</code>	DPO, RL

Algorithm 2 Creation of the MENTOR-derived module training corpus

Require: dendrogram D , multiplex M , difficulty map δ

```
1: Parse the genome-wide MENTOR dendrogram  $D$  over multiplex  $M$  into modules  
    $\{C_{\text{module}}\}$   
2: Partition modules into train/validation/test splits at the module level  
3: for each module  $C_{\text{module}}$  in split with difficulty map  $\delta$  do  
4:    $n \leftarrow |C_{\text{module}}|$   
5:   // explanation  
6:   emit ( $C^{\text{in}} = C_{\text{module}}$ ,  $C = C_{\text{module}}$ , explanation,  $rel = \text{validated\_group}$ )  
7:   // recovery  
8:    $k_{\text{drop}} \leftarrow \text{DROPCOUNT}(n, \delta_{\text{rec}})$ ;  $G_{\text{drop}} \leftarrow \text{DETSELECT}(C_{\text{module}}, k_{\text{drop}})$   
9:   emit ( $C^{\text{in}} = C_{\text{module}} \setminus G_{\text{drop}}$ ,  $C = C_{\text{module}}$ , recovery,  $rel = \text{validated\_group}$ )  
10:  // refinement  
11:   $k_{\text{add}} \leftarrow \text{NOISECOUNT}(n, \delta_{\text{ref}})$ ;  $G_{\text{add}} \leftarrow \text{DENDRONEG}(C_{\text{module}}, k_{\text{add}}, \delta_{\text{ref}})$   
12:  emit ( $C^{\text{in}} = C_{\text{module}} \cup G_{\text{add}}$ ,  $C = C_{\text{module}}$ , refinement,  $rel =$   
    validated\_group)  
13:  // none  
14:   $G_{\text{none}} \leftarrow \text{DENDRONEG}(C_{\text{module}}, n, \delta_{\text{none}}, \text{conflict\_free})$   
15:  emit ( $C^{\text{in}} = G_{\text{none}}$ ,  $C = \emptyset$ , none,  $rel = \text{insufficient\_support}$ )  
16: end for  
17: return all task instances
```

other gene in the network. We then apply the RWR++ elbow filter to the score-versus-rank curve for that seed, yielding an adaptive cutoff rank r_g^* . The LOE module is defined as $C_g^{\text{loe}} = \{g\} \cup \{v : \text{rank}_g(v) \leq r_g^*\}$, producing a single seed-specific module per gene without imposing a fixed module size.

For each module, we materialize the same four task instances as the MENTOR-derived corpus. For difficulty control, we substitute RWR-LOE rank bands for dendrogram distance bands: noise and none genes are drawn from ranks just beyond, moderately beyond, or far beyond the seed’s elbow cutoff. We partition RWR-LOE modules into train, validation, and test splits independently of the MENTOR-derived modules, and module-size bins are assigned after construction from the realized values of $|C_g^{\text{loe}}|$. The two corpora are complementary: MENTOR modules capture top-down, hierarchical functional organization, while RWR-LOE modules reflect bottom-up, seed-centered topology.

Graph Environment and Runtime

Each task instance is paired with a graph environment defined over the same canonical gene universe as the in-house brain multiplex, from which our module corpora are derived. The sampled input set C^{in} initializes the agent’s graph anchors, while the hidden target C defines the supervisory signal.

Algorithm 3 Creation of the RWR-LOE module training corpus

Require: multiplex M , gene universe $\mathcal{G}$, elbow-filter parameters ϕ , difficulty map δ

```
1: Partition  $\mathcal{G}$  into train/validation/test splits at the gene level
2: for each gene  $g \in \mathcal{G}$  do
3:   Run rwr_loe( $g, M$ ) to obtain scores  $s_g$  and ranked list  $L_g$ 
4:    $r_g^* \leftarrow \text{ELBOWCUTOFF}(L_g, s_g, \phi)$ 
5:    $E_g \leftarrow \{v \in L_g : \text{rank}_g(v) \leq r_g^*\}$ 
6:    $C_g^{\text{loe}} \leftarrow \{g\} \cup E_g, \quad n \leftarrow |C_g^{\text{loe}}|$ 
7:   // explanation
8:   emit ( $C^{\text{in}} = C_g^{\text{loe}}, C = C_g^{\text{loe}}, \text{explanation}, \text{rel} = \text{validated\_group}$ )
9:   // recovery
10:   $k_{\text{drop}} \leftarrow \text{DROPCOUNT}(n, \delta_{\text{rec}}); \quad G_{\text{drop}} \leftarrow \text{DETSELECT}(C_g^{\text{loe}}, k_{\text{drop}})$ 
11:  emit ( $C^{\text{in}} = C_g^{\text{loe}} \setminus G_{\text{drop}}, C = C_g^{\text{loe}}, \text{recovery}, \text{rel} = \text{validated\_group}$ )
12:  // refinement
13:   $k_{\text{add}} \leftarrow \text{NOISECOUNT}(n, \delta_{\text{ref}}); \quad G_{\text{add}} \leftarrow \text{RANKBANDNEG}(L_g, r_g^*, k_{\text{add}}, \delta_{\text{ref}})$ 
14:  emit ( $C^{\text{in}} = C_g^{\text{loe}} \cup G_{\text{add}}, C = C_g^{\text{loe}}, \text{refinement}, \text{rel} = \text{validated\_group}$ )
15:  // none
16:   $G_{\text{none}} \leftarrow \text{RANKBANDNEG}(L_g, r_g^*, n, \delta_{\text{none}}, \text{conflict\_free})$ 
17:  emit ( $C^{\text{in}} = G_{\text{none}}, C = \emptyset, \text{none}, \text{rel} = \text{insufficient\_support}$ )
18: end for
19: return all task instances
```

To interface the graph environment with the training pipeline, we use a compiled native multiplex library exposed to Python through `ctypes` bindings [20]. Training, rollout, and optimization code are implemented in Python [38], while graph storage and graph operations are handled by the native runtime for efficient execution on large multiplexes. Multiplex networks are persisted as raw, CSR-style binary stores described by JSON manifests; all graph input and output is binary, with the manifest carrying the layer names, node, and build metadata needed to load the graph.

The runtime uses a set of optimized multiplex functions developed as part of the MENTOR [44] and MENTOR-IA [47] projects known as RWR++. Core operators include neighborhood retrieval, shortest-path search, random walk with restart on both monoplex and multiplex views, and subgraph induction. These operators provide the primary graph-facing actions available to the agent during exploration.

All runtime operators return both graph objects and provenance metadata. In addition to nodes, edges, and scores, each returned object records the relevant layer identity, operator parameters, and source information needed to trace the final interpretation back to the graph evidence used to construct it. To ensure consistency across serialized multiplex files, C++ runtime objects, Python wrappers, and logged training artifacts, all nodes are represented in a single canonical identifier space and all edges are referenced using deterministic, layer-aware identifiers.

0.2.7 Trajectory Generation, Preference Mining, and Mechanistic Summary Construction

Overview

Using the module corpora and deterministic runtime defined in Sections 0.2.6, 0.2.6, and 0.2.6, we generate shared-prefix trajectories for SFT and DPO. For each sampled task, the warm-start policy proposes multiple candidate next steps, deterministic execution produces candidate observations and next states. We employ structured branch scoring functions to rank these candidates, and the resulting rankings are converted into shared-prefix preference pairs. One branch per step is retained as the continuation of the logged trajectory, from which we derive per-step findings and a final mechanistic summary. The trajectory generation process is outlined in Algorithm 4.

Shared-prefix Trajectory Synthesis

Given a sampled task and initialized state, we sample multiple action candidates $(r_{t,i}, a_{t,i})$ given the current decision context D_t and execute valid tool calls in the deterministic graph runtime. To ensure that candidates with the same prefix explore genuinely distinct regions of the action space, we use verbalized sampling based on the task type. For example, recovery prompts bias candidates towards random-walk expansion, neighborhood expansion, subgraph validation, or a direct decision, while insufficient evidence (`none`) prompts direct the model towards disconfirming graph evidence. When a sampling round for recovery or refinement tasks fails to produce valid tool calls, we issue a tool-coverage retry that requires a valid graph operator whenever one can be formed, which prevents the branch pool from degenerating into tool-free continuations. Conditioned on each candidate action and observation, the same policy then samples multiple verification candidates, yielding a tree of candidate next states rooted at a shared prefix. The branched sampling scheme is visualized in Figure 2.

Initialization, Tool Validation, and Query Construction

We define $\text{InitState}(q, e^{\text{user}}, C^{\text{in}})$ as a deterministic function that constructs the initial interpretation I_0 and structured state S_0 from the query, user-provided evidence, and seed gene set. The working interpretation I_0 is populated from a fixed template: the mechanistic claim is empty, the evidence field records only the user-provided inputs, the uncertainty field is blank, and the next subgoal is set to the query q . The structured state S_0 is initialized as follows:

1. the user-provided anchors are set to (q, e^{user}) ,
2. the relationship status rel_0 is set to `unknown`,

Algorithm 4 Trajectory Generation for SFT and DPO

Require: $\pi_\theta^{act}, \pi_\theta^{verify}$, runtime ε , $T_{\max}, n_{act}, n_{ver}, n_{cov}$, directives $\{d_i\}$, pair margin τ , selection tolerance ϵ .

Sample module task $(C^{\text{in}}, C, C_{\text{type}})$ and evidence mode ρ .

$q, e^{\text{user}} \sim Q_{\text{templates}}(\cdot \mid C^{\text{in}}, C, C_{\text{type}}, \rho)$.

$(I_0, S_0) \leftarrow \text{InitState}(q, e^{\text{user}}, C^{\text{in}})$.

$\tau^* \leftarrow [], \mathcal{P} \leftarrow [], \mathcal{F} \leftarrow [], T \leftarrow 0$

$z_{-1} \leftarrow \text{continue}$

for $t = 0, 1, \dots, T_{\max} - 1$ **do**

if $z_{t-1} = \text{stop}$ **or** $\text{budget}(S_t) \leq 0$ **then break**

end if

$D_t \leftarrow (q, e^{\text{user}}, I_t, S_t), \mathcal{C}_t \leftarrow [], \mathcal{A} \leftarrow []$

for $i = 0, \dots, n_{act} - 1$ **do**

$(r_{t,i}, a_{t,i}) \sim \pi_\theta^{act}(\cdot \mid D_t, d_i)$; append to $\mathcal{A}$

end for

if $C_{\text{type}} \in \{\text{recovery}, \text{refinement}\}$ **and** no $a \in \mathcal{A}$ is a tool call **then**

for $i = 0, \dots, n_{cov} - 1$ **do**

$(r, a) \sim \pi_\theta^{act}(\cdot \mid D_t, d_i^{\text{cov}})$; append to $\mathcal{A}$

end for

end if

for each $(r_{t,i}, a_{t,i}) \in \mathcal{A}$ **do**

$o_{t,i} \leftarrow \varepsilon(a_{t,i})$ **if** $\text{ValidTool}(a_{t,i}, S_t)$ **else** $\emptyset$

for $j = 0, \dots, n_{ver} - 1$ **do**

$(I'_{t,i,j}, S'_{t,i,j}, z'_{t,i,j}) \sim \pi_\theta^{verify}(\cdot \mid D_t, r_{t,i}, a_{t,i}, o_{t,i})$

$s_{t,i,j}^{\text{local}} \leftarrow \text{LocalScore}(D_t, I'_{t,i,j}, S'_{t,i,j}, r_{t,i}, a_{t,i}, o_{t,i}, z'_{t,i,j}; C, C_{\text{type}})$

 Append $c_{t,i,j}$ to $\mathcal{C}_t$

end for

end for

$\text{NormalizeScores}(\mathcal{C}_t)$

$(i^*, j^*) \leftarrow \text{SelectBranch}(\mathcal{C}_t; \epsilon)$

$\mathcal{P} \leftarrow \mathcal{P} \cup \text{MakePreferencePairs}(\mathcal{C}_t, c_{t,i^*,j^*}, \tau)$

 Append $(I_t, S_t, c_{t,i^*,j^*})$ to τ^* ; $f_t \leftarrow \text{RenderFinding}(\cdot)$; append f_t to $\mathcal{F}$

$(I_{t+1}, S_{t+1}) \leftarrow (I'_{t,i^*,j^*}, S'_{t,i^*,j^*})$; $T \leftarrow t + 1$

end for

$m \leftarrow \text{RenderSummary}(S_T, \mathcal{F})$

return $\tau^*, \mathcal{F}, m, \mathcal{P}$

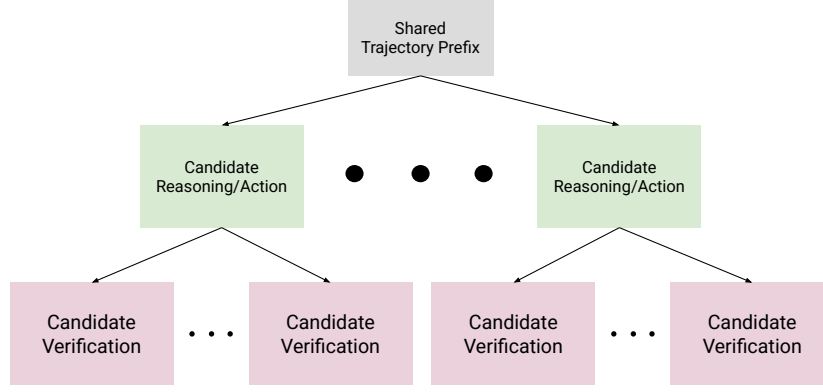


Figure 2: The shared prefix generation tree.

3. the predicted group is initialized to $G_0 = C^{\text{in}}$,
4. the evidence log E_0 is empty,
5. the predicted mechanistic labels are empty,
6. the remaining budget is T_{max} and the continuation state is $z_0 = \text{continue}$.

The agent does not observe the task type C_{type} , the true relationship status, or the hidden target C ; it must infer the appropriate task case through exploration.

We define $\text{ValidTool}(a_{t,i}, S_t)$ as a deterministic function that checks whether a proposed tool action $a_{t,i}$ is executable given the current structured state S_t . Validation proceeds in two stages. First, the syntax check verifies that the tool name referenced in $a_{t,i}$ exists in the current tool vocabulary and that all arguments conform to the tool’s input schema. Second, the semantic check verifies that basic conditions are satisfied with respect to the graph environment. For example, the function ensures that queried genes exist in the multiplex, requested layers are valid, and required fields in S_t are populated. If either check fails, ValidTool returns false and the observation

$o_{t,i}$ is set to $\emptyset$. Invalid tool calls are counted toward n_{inv} in the efficiency penalty (Equation 5).

At inference time, users provide a natural language query q alongside optional evidence e^{user} . Queries are unconstrained and may range from broad exploratory requests to specific hypothesis-driven questions. Evidence may include any combination of:

1. a seed gene list,
2. a multiplex graph or subgraph,
3. free-text context such as experimental notes or prior findings, and
4. structured annotations such as GO terms, pathway labels, or expression data from prior analysis.

During training, we simulate this variability by generating diverse queries and evidence configurations from module metadata. For each sampled task $(C^{\text{in}}, C, C_{\text{type}})$, we sample a query from task-type-specific deterministic templates and an evidence mode $\rho \in \{\text{minimal}, \text{graph}\}$ that controls which evidence types are provided to the agent. Table 2 defines representative query templates and compatible evidence modes for each task type. To promote phrasing diversity, we use an LLM to paraphrase each template query into natural language variants conditioned on the gene symbols of the sampled module. All generated queries and evidence configurations are cached during dataset construction for reproducibility.

Table 2: Representative query templates and compatible evidence modes by task type. Evidence modes: **minimal** (seed gene list) or **graph** (seed gene list + multiplex subgraph).

Task type	Example query	Evidence modes
recovery	“What other genes are functionally related to {seed genes}?”	minimal, graph
refinement	“Which of these genes do not belong together?”	minimal, graph
explanation	“What mechanism connects {seed genes}?”	minimal, graph
none	“Are these genes mechanistically related?”	minimal, graph

Branch Ranking, Preference Mining, and Selection

Each candidate branch is assigned a deterministic local score computed from the structured state, runtime outputs, and hidden module targets. These local scores are used to rank same-prefix candidates and mine DPO preference pairs against that selection. Because several branches frequently tie or fall within a small margin of each

other, continuation selection does not reduce to a strict $\arg \max$, but instead, within a tolerance margin ϵ of the best normalized score, ties are broken with task-specific heuristics such as prompted seed gene retention and tool-supported evidence, pushing the logged trajectory to favor well-grounded candidates over those that score highly by chance. We defer the formal score definitions to Section 0.2.8.

0.2.8 Preference Scoring and Reward Design

Design Principles and Scoreable Objects

We define all training-time scores over structured state transitions, deterministic runtime metadata, and hidden module-derived targets rather than textual reasoning. This design choice keeps scoring reproducible, machine-parseable, and scalable, while avoiding dependence on human judges or semantic evaluation of natural-language trajectories. We decompose all scores into four components: schema compliance, group membership, mechanistic quality, and efficiency.

Deterministic Local Branch Scoring for DPO

We define local branch scoring for DPO as

$$s_{t,i,j}^{local} = w_s s^{schema} + w_c \Delta s^{module} + w_m \Delta s^{mech} - w_e p^{eff} - p^{mismatch} \quad (1)$$

where Δs^{mech} is the evidence-grounded mechanistic quality delta. Additionally, mechanistic quality is clamped to be non-positive when the candidate degrades the predicted group relative to the task type (e.g. by adding noise genes during recovery or removing true members during refinement), and $p^{mismatch}$ penalizes an **explanation** candidate that reduces recovery of an already-complete module.

We define $s_{t,i,j}^{schema}$ as a binary indicator that equals 1 if the candidate’s structured state and interpretation updates are schema valid, and 0 otherwise. Specifically, schema compliance is defined by a properly-formed JSON with all required fields present and tool calls referencing valid tool names and arguments. We formalize this term as

$$s_{t,i,j}^{schema} = \mathbf{1}(\text{SchemaValid}(c_{t,i,j})). \quad (2)$$

We use Δs^{module} to measure the change in module membership accuracy from step t to $t + 1$. Because the runtime state may contain multiple predicted groups, we first score a state by the best-matched predicted group:

$$s^{module}(z; C, C_{type}) = \max_{G \in \mathcal{G}(z)} [\alpha J(G, C) + \beta P(G, C) + \gamma R(G, C)], \quad (3)$$

where $\mathcal{G}(z)$ denotes the set of predicted groups in state z , with an empty group used when no prediction is present. We then define the local module improvement as

$$\Delta s_{t,i,j}^{module} = s^{module}(z_{t+1,i,j}; C, C_{type}) - s^{module}(z_t; C, C_{type}). \quad (4)$$

where J , P , and R denote Jaccard, precision, and recall respectively, the weights α, β, γ are set per task type. Recovery upweights recall ($\alpha, \beta, \gamma = 0.2, 0.2, 0.6$) to reward found members, refinement upweights precision ($0.2, 0.6, 0.2$) to penalize retained noise, and explanation weights them equally ($\frac{1}{3}$ each). This best-group formulation supports the multiple-groups state representation while keeping the hidden target single-module for scoring.

We define the mechanism quality of a state as an evidence-grounded score $m_t \in [0, 1]$. From the evidence log, we extract three support signals: enrichment support e_t , annotation support g_t (from `query_mygene`), and network support u_t , each $\in [0, 1]$. We define a consensus term $\kappa_t = \max(e_t, g_t)$, a cross-tool agreement term $\chi_t = \min(e_t, g_t)$, an evidence strength term $\sigma_t = \max(e_t, 0.75 g_t, 0.5 u_t)$, and an annotation-only strength term $\sigma_t^{\text{ann}} = \max(e_t, 0.75 g_t)$. We additionally compute a grounding term γ_t (claim entities are supported by observed evidence), a specificity term ψ_t (vague or generic claims are penalized), a stability term b_t (enrichment consistency across query subsets), and a discounted network-agreement term $\hat{u}_t = u_t$ if $\kappa_t > 0$ and $0.35 u_t$ otherwise. An abstention term α_t rewards withholding a claim under weak support: $\alpha_t = 1 - \sigma_t$ when the state abstains (relationship status is `insufficient_support` or an abstaining claim), 0 when an unsupported claim is asserted ($\sigma_t \leq 0.05$), and 0.5 otherwise.

The score takes one of two branches depending on whether the state abstains:

$$m_t = \begin{cases} 0.60 \alpha_t + 0.20 \hat{u}_t + 0.20 \gamma_t & (\text{abstaining}) \\ 0.25 \gamma_t + 0.25 \kappa_t + 0.18 \psi_t + 0.12 \hat{u}_t + 0.10 b_t + 0.10 \chi_t & (\text{asserting}) \end{cases}$$

In the asserting branch, the score is multiplied by 0.35 when a claim or labels are present but unsupported ($\sigma_t \leq 0.05$). In the abstaining branch m_t is forced to 0 for any non-`none` task whose annotation evidence is absent ($\sigma_t^{\text{ann}} \leq 0.05$), preventing weak-evidence abstention from being rewarded on tasks that should produce a grounded group. The per-step mechanistic reward is the change $\Delta s_{t,i,j}^{\text{mech}} = m_{t+1,i,j} - m_{t,i,j}$, set to 0 if no mechanistic claim has been made.

p_{eff} is a small penalty designed to discourage two behaviors: excessively long trajectories and invalid tool calling. Long trajectories are penalized via the term t/T_{max} , while we count invalid tool calls (those that are schema-invalid, exact duplicates, or return empty observations) with n_{inv} , penalizing them proportionally to the total number of tool calls: ($n_{\text{inv}}/n_{\text{total}}$). We formalize this scoring term as

$$p_{t,i,j}^{\text{eff}} = \lambda_1(t/T_{\text{max}}) + \lambda_2(n_{\text{inv}}/n_{\text{total}}). \quad (5)$$

For the `none` task type, where the hidden target is empty, the membership component is replaced by an abstention score

$$s^{\text{none}} = w_{\text{rel}} \mathbf{1}[\text{rel} = \text{insufficient_support}] + w_{\text{abs}} \mathbf{1}[G = \emptyset],$$

with $w_{\text{rel}} = 0.6$ and $w_{\text{abs}} = 0.4$. This rewards the agent both for declaring no shared mechanism and for refusing to emit a spurious group.

Terminal Reward Decomposition for GRPO

Unlike DPO, which compares local same-prefix branches, policy gradient methods require a scalar reward for a completed rollout. We therefore define a terminal reward over the final structured state and trajectory log:

$$R_T = w_s^T s_{schema}^T + w_{abs}^T s_{module}^T + w_c^T \Delta s_{module}^T + w_{mabs}^T s_{mech}^T + w_m^T \Delta s_{mech}^T - w_e^T p_{eff}^T. \quad (6)$$

While all terms included in local scoring appear in the terminal reward, they are modified to capture global behavior. Additionally, two new rewards are included to capture absolute recovery. Absolute terms appear in the terminal score but not the local score. Because local scoring is evaluated on examples that share the same starting state S_t , absolute levels cancel, and only relative metrics matter. Conversely, the terminal reward is evaluating complete trajectories, which may vary in starting difficulty and length, so absolute metrics capture global problem-solving ability.

We define the absolute module membership score as the composite Jaccard, precision, and recall evaluated on the final predicted group G_T against the hidden target C :

$$s_{module}^T = \alpha J(G_T, C) + \beta P(G_T, C) + \gamma R(G_T, C), \quad (7)$$

using the same weighting coefficients α, β, γ defined in Equation 4. The cumulative module delta is the difference between the final and initial composite scores:

$$\Delta s_{module}^T = s_{module}^T - s_{module}^0, \quad (8)$$

where $s_{module}^0 = \alpha J(G_0, C) + \beta P(G_0, C) + \gamma R(G_0, C)$ is the composite score at initialization.

The absolute mechanism score is the evidence-grounded mechanism quality m_T of the final state:

$$s_{mech}^T = m_T. \quad (9)$$

The cumulative mechanistic delta is the corresponding difference:

$$\Delta s_{mech}^T = m_T - m_0. \quad (10)$$

If no mechanistic claim has been made at termination, we set $s_{mech}^T = 0$ and $\Delta s_{mech}^T = 0$.

The terminal schema score checks that the final structured state is schema-valid and that the termination mode is properly recorded:

$$s_{schema}^T = \mathbf{1}(\text{SchemaValid}(S_T) \wedge \text{TerminationRecorded}(S_T)). \quad (11)$$

Finally, the terminal efficiency penalty aggregates trajectory length and invalid tool use across all steps:

$$p_{eff}^T = \lambda_1 \left(\frac{T}{T_{\max}} \right) + \lambda_2 \left(\frac{\sum_{t=0}^{T-1} n_{inv}^t}{\sum_{t=0}^{T-1} n_{total}^t} \right). \quad (12)$$

Preference Pair Construction, Score Normalization, and Filtering

At each decision point t , we create DPO preference tuples of the form (x_t, c_t^+, c_t^-) from our candidate set $\mathcal{C}_t = \{c_{t,i,j}\}$. We define $x_t = (q, e^{\text{user}}, I_t, S_t)$ as the shared-prefix context and $c_{t,i,j}$ as a sample interpretation continuation from Algorithm 4. Each continuation contains the model-generated reasoning step, tool action, verifier update, and continuation decision.

Before constructing pairs, we normalize scores to ensure that the selection margin has a consistent meaning across task types and trajectory lengths. We apply min-max normalization within each candidate pool $\mathcal{C}_t$, rescaling the composite local scores to $[0, 1]$. We denote the normalized score of candidate $c_{t,i,j}$ as $\bar{s}_{t,i,j}^{\text{local}}$.

To create paired examples, we set the preferred candidate c_t^+ to the branch selected as the trajectory continuation, c_{t,i^*,j^*} (Section 0.2.7), rather than to a strict arg max of the normalized score. Because selection resolves near-ties with task-specific heuristics within a tolerance ϵ , the preferred branch is the one judged best for the task, not merely the highest-scoring candidate. We then define a selection margin τ and drop all candidates for which

$$\bar{s}_{t,i^*,j^*}^{\text{local}} - \bar{s}_{t,i,j}^{\text{local}} < \tau,$$

where (i^*, j^*) indexes the selected candidate. This prevents the construction of pairs that are near ties. The remaining candidates form the eligible dispreferred pool $\mathcal{C}_t^-$.

From $\mathcal{C}_t^-$, we draw dispreferred examples across three difficulty bins, $d \in \{\text{easy}, \text{medium}, \text{hard}\}$. We construct **easy** pairs by selecting the lowest-scoring candidate in $\mathcal{C}_t^-$, **medium** pairs by selecting the candidate with the median score, and **hard** pairs by selecting the highest-scoring candidate in $\mathcal{C}_t^-$.

We store each preference pair (x_t, c_t^+, c_t^-) along with its difficulty bin d , task type C_{type} , decision step t , raw and normalized scores, and the score margin in the preference corpus $\mathcal{P}$. After trajectory generation is complete, we rebalance $\mathcal{P}$ across task types and difficulty bins by downsampling overrepresented categories to ensure that the training distribution is not dominated by task types that produce disproportionately many informative pairs. The curriculum strategy described in Section 0.2.10 controls which difficulty bins are included at each training stage.

Mechanistic Rendering and Training Outputs

Each selected trajectory is logged as a sequence of turn records, from which we render short per-step findings and a final mechanistic summary. These artifacts are then converted into SFT trajectory examples, evidence-to-finding records, and DPO preference pairs for the downstream training pipeline. Additionally, scored trajectories can be utilized for policy gradient optimization, such as PPO or GRPO.

0.2.9 Training Pipeline

We propose a training pipeline that closely aligns with standard model post-training pipelines. First, we utilize supervised fine-tuning for general biological knowledge and tool calling. We continue training with DPO on stepwise interpretation tree comparisons. Finally, we promote novel exploration over long-horizon tasks with policy gradient optimization.

Warm-Start SFT

We first train our policy model with warm-start SFT using the closed-book dataset outlined in Section 0.2.6. We will train our policy model on OLCF’s Frontier Supercomputer [3], utilizing slurm [53] for job scheduling, huggingface and TRL [50] for weight updating, DeepSpeed ZeRO-3 for model parallelism [37], Low-Rank Adaptation for parameter-efficient fine-tuning [18], Weights and Biases [7] for tracking convergence, and gpt-oss-120b [32] as our base policy. We will utilize early stopping checkpointing to prevent model overtraining.

Tool-Call and Evidence-to-Finding SFT

We continue the supervised fine-tuning stage by adding open-book tool calling examples and evidence-to-finding examples to our data mixture. We initialize our policy from the warm-start SFT checkpoint with the lowest loss, and we utilize the same pipeline for model training.

DPO Training

After training our model via SFT, we will utilize its learned domain knowledge and tool calling capabilities to perform DPO training. We will utilize examples generated with the trajectory construction methods specified in 0.2.7 to create paired preference data. Branch continuations will be scored using the functions defined in Section 0.2.8. At each decision point, the branch selected as the trajectory continuation serves as the preferred candidate c_t^+ , while dispreferred candidates c_t^- are drawn from the eligible dispreferred pool across the **easy**, **medium**, and **hard** difficulty bins. Model weights will be optimized using the DPO objective function [36]:

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right].$$

Additionally, during training, we will generate trajectories with new, trained model checkpoints to improve our model’s generated preference data. This will allow us to continue making model improvements even after scoring is optimized on our initial dataset.

Online RL for Long-Horizon Exploration

After training with DPO, we will implement a final GRPO stage for long-horizon exploration. Unlike DPO, GRPO requires no annotated trajectories; rather, optimization is done by calculating the advantage of an output relative to the rollout group $\{o_1, o_2, \dots, o_G\}$ using the function

$$\mathcal{J}_{GRPO} = \mathbb{E}[q \sim P(Q), \{y_i\}_{i=1}^G \sim \pi_{\theta_{old}}(Y \mid q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \left\{ \min \left[\frac{\pi_{\theta}(y_{i,t} \mid q, y_{i,<t})}{\pi_{\theta_{old}}(y_{i,t} \mid q, y_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(y_{i,t} \mid q, y_{i,<t})}{\pi_{\theta_{old}}(y_{i,t} \mid q, y_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] \right. \\ \left. - \beta \mathbb{D}_{KL}[\pi_{\theta} \parallel \pi_{ref}] \right\},$$

where

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{R_i^T - \text{mean}(R^T)}{\text{std}(R^T)}$$

By foregoing structured, annotated preference pairs, GRPO promotes novel exploration for problem solving. We aim to utilize this policy gradient method to prevent mode collapse in exploration trajectories and reinforce full coverage of the exploration space. The reference policy π_{ref} is fixed at the DPO checkpoint that initializes GRPO and is not updated during online rollouts, so the KL term anchors exploration to a stable reference rather than drifting with the trained policy.

0.2.10 Curriculum and Optimization Strategy

We will employ a training curriculum throughout the optimization process, scaling problem difficulty with model performance.

During the SFT stage, we scale difficulty by moving from closed-book biological questions to open-book biological questions that require tool use. This promotes learning biological understanding first, and then moving to structured, schema-respecting tool calling tasks second.

After training a joint closed- and open-book SFT checkpoint, we scale difficulty for DPO pairs by training on increasingly difficult examples, based on difficulty bins defined in Section 0.2.8. We begin DPO training by limiting preference pairs to **easy** examples, then adding **medium** and **hard** examples as training saturates for each difficulty. After DPO training converges on **hard** preference pairs, we transition to online RL to promote exploration beyond the behaviors captured in logged trajectory outputs.

We scale long-horizon reinforcement learning difficulty with several strategies. First, we modulate the trajectory length $T_{\max}$, starting with shorter exploration

budgets and gradually increasing exploration length for problems that require multi-hop exploration. Additionally, we modulate realized module size, beginning training on small modules and shifting to larger ones as rewards saturate. Finally, we scale the per-task difficulty level, which jointly controls the number of dropped and added genes and the source-specific distractor band from which noise and **none** genes are drawn: dendrogram-distance bands for MENTOR-derived modules (Section 0.2.6) and post-elbow rank bands for RWR-LOE modules (Section 0.2.6). Higher difficulties remove or add more genes and draw distractors from nearer, more easily confused regions of the corresponding module source, making recovery and refinement harder. At each stage, training batches are sampled from both the MENTOR-derived and RWR-LOE module pools and are rebalanced by task type and difficulty bin to prevent either corpus from dominating training.

0.2.11 Evaluation and Benchmarks

Module Holdout Evaluation

We will use several metrics to evaluate the effectiveness of our model. Each metric will be calculated by parsing the model’s terminal structured state S_T , allowing for scalable evaluation of our checkpoints. All evaluations will be conducted on a held out set of MENTOR- and RWR-LOE-derived modules from the brain multiplex.

We will report four metrics for evaluation: Jaccard index J , precision P , recall R , and F_1 score on the predicted module C^{pred} vs. the hidden module target C . Jaccard index is defined as

$$J(C^{pred}, C) = \frac{|C^{pred} \cap C|}{|C^{pred} \cup C|},$$

penalizing both missing and extra entities. Precision is defined as

$$P = \frac{TP}{TP + FP},$$

measuring the proportion of true positive entities compared to all predicted positives. Recall is defined as

$$R = \frac{TP}{TP + FN},$$

measuring the proportion of true positive predictions to the sum of true positives and false negatives. The F1 score is defined as the harmonic mean of precision and recall,

$$F1 = \frac{2}{\frac{1}{P} + \frac{1}{R}},$$

providing a balanced view of precision and recall to measure general performance. In addition to these module recovery metrics, we will measure mechanistic grounding quality, defined in Equation 9, to assess whether mechanistic claims are supported by retrieved evidence.

We will report the weighted composite reward R_T , defined in Section 0.2.8, to directly compare training reward magnitude on validation and holdout splits. If the task type is **none**, meaning $C = \emptyset$, we will parse the S_T for the predicted relationship rel_T . Along with computing global accuracy, we will distinguish between both error directions, analyzing when the model claims no mechanism exists for truly related gene sets and when the model fabricates a mechanism for truly unrelated entities.

Furthermore, we will report efficiency metrics, such as mean trajectory length, budget utilization, and invalid tool call rate. This allows for comparison between holdout efficiency and validation efficiency metrics computed in Equation 12.

In addition to general evaluation, we will stratify our evaluation across several axes: module size, noise, task type, and module source. We will divide module size and noise into three discrete bins each for comparative evaluation. Reporting stratified metrics allows us to analyze how MENTOR-RL performs as task difficulty increases and task type changes, and comparing metrics by module source exposes whether the model generalizes across module construction methods or exhibits biases toward one of the two module types.

Novel-Domain Evaluation

Because the holdout split above is drawn from the same brain multiplex used for training, it measures in-distribution generalization. To assess out-of-distribution performance, we additionally evaluate on MENTOR-derived modules from independently published multiplex studies the model was not trained on, such as networks built for opioid use disorder and cardiac studies. These modules are produced by the same MENTOR dendrogram procedure (Section 0.2.6) over different tissue- and disease-specific multiplexes, so they share the module-recovery scoring methodology while presenting genuinely novel network context. We report the same metrics defined in Section 0.2.11 and expect a performance gap relative to the brain-multiplex holdout, reflecting distribution shift. Because these modules derive from real experimental networks with expert- and algorithm-verified structure, they let us characterize the strengths and limitations of MENTOR-RL on real-world, out-of-distribution data.

Blind Comparative Evaluation

In conjunction with empirical validation, which measures model accuracy, we will conduct blind comparative evaluations to measure expert domain scientists’ model preferences. First, we will perform an inter-model preference test, supplying interpretation outputs from MENTOR-RL, GPT-5 [43], gpt-oss-120b [32], LLaMA 4 [2], Kimi K2 [46], and Nemotron 3 Super [31]. We will provide between 5 and 15 domain experts with paired samples containing MENTOR-RL outputs and outputs from another randomly selected model. We will then ask scientists to indicate their preferred interpretation, and with this information, we will calculate the proportion of MENTOR-RL outputs that are preferred compared to other models. To support

interpretation of these proportions, we will overlap a subset of evaluation pairs across raters and report inter-expert rating reliability.

After conducting inter-model preference testing, we will perform intra-model preference testing to compare preferences for different model checkpoints. Using the same protocol described for inter-model comparative evaluation, we will compare the base, SFT, DPO, and GRPO-trained MENTOR-RL checkpoints. Blind comparative testing will allow us to evaluate model characteristics such as style, tone, and concision that are not easily evaluable with empirical metrics.

Efficiency and Ablations

To ensure that each component of our methodology contributes to an improvement in mechanistic interpretation, we will ablate both training stages and design elements. All ablations will be evaluated using the holdout metrics defined in Section 0.2.11.

We will ablate phases of our training pipeline to confirm that each stage improves over the previous checkpoints. First, we will test our SFT-only checkpoint compared to the SFT- and DPO-trained checkpoint to ensure that preference optimization yields interpretation gains. Next, we will compare the DPO checkpoint to the full training pipeline to determine whether long-horizon RL training leads to improvement over local preference training. Finally, we will examine the effects of training with only closed-book SFT data vs. a mixed closed- and open-book training set to evaluate whether tool-calling examples in SFT lead to better DPO optimization.

Along with testing various training stages, we will ablate architectural and reward decisions. First, we will ablate the policy’s verifier mode, removing it to test whether self-verification is useful for producing better results. Additionally, we will remove the efficiency penalty p^{eff} from the reward to test whether MENTOR-RL can learn reasonable trajectory lengths without regularization. Finally, we will ablate difficulty bins in our training curriculum, training on all difficulty bins simultaneously rather than staging them progressively. This ablation will allow us to examine whether the model can learn from hard examples without curriculum staging.

0.3 Expected Results and Deliverables

0.3.1 Expected Empirical Results

We expect MENTOR-RL to recover held-out MENTOR-derived and RWR-LOE modules with a composite F_1 competitive with frontier models at substantially lower per-query inference cost while improving monotonically across the training pipeline. Each of the SFT, DPO, and GRPO checkpoints should outperform the preceding one on module recovery and mechanistic grounding. Under the stratified evaluation defined in Section 0.2.11, we expect the largest gains on **recovery** and **refinement** tasks, where deterministic graph operators provide dense supervisory signals, and smaller but nontrivial gains on **explanation** and **none** tasks, where reward is driven

by mechanistic grounding and abstention rather than set-level recovery. On the novel-domain evaluation against experimentally-derived MENTOR modules, we expect a performance gap relative to brain multiplex holdout, reflecting distributional shift, but with the gap narrowing as curriculum difficulty and noise levels increase during training. We expect comparable F_1 on held-out RWR-LOE modules relative to MENTOR-derived modules given the shared task structure, with any gap reflecting differences in module coherence between the two construction methods.

0.3.2 Expected Behavioral Results

Beyond empirical accuracy, we expect three behavioral outcomes. First, the act-verify loop should reduce schema-invalid tool calls and unsupported mechanistic claims relative to a verifier-ablated baseline, measured by invalid tool call rate and ungrounded-claim rate on the holdout split. Second, the efficiency penalty should produce shorter average trajectories without loss of recovery accuracy relative to a penalty-ablated baseline. Third, blind comparative evaluation should show that expert scientists prefer MENTOR-RL outputs to base-policy outputs at a significantly higher rate, and preferences relative to frontier models should improve across successive training stages.

0.3.3 Deliverables

We will release, under licenses compatible with the underlying data sources, the following artifacts:

1. the module-grounded training environment, including the `ctypes` multiplex runtime, tool vocabulary, and deterministic scoring functions,
2. train, validation, and test split indices at the module level, together with the code required to reconstruct the raw training corpus and cached API responses,
3. trained MENTOR-RL checkpoints for each stage of the pipeline (warm-start SFT, joint closed- and open-book SFT, DPO, and GRPO),
4. the evaluation harness, including MENTOR module holdout benchmarks, out-of-distribution published multiplex benchmarks, and blind comparative evaluation protocols, and
5. training and generation code, curriculum configurations, and logged trajectories to support reproducible re-training and ablation.

0.3.4 Broader Impact

By releasing a deterministically-scored, open-source mechanistic exploration agent together with its training environment, we aim to lower the cost of biological

interpretation research and establish a reproducible benchmark that does not require proprietary model access. The module-grounded scoring framework is domain-specific, but the underlying pattern, deterministic structured-state rewards over a tool-using policy, is transferable to other scientific settings with curated ground truth.

0.4 Discussion and Conclusion

0.4.1 Reproducibility Without Proprietary Judges

The training pipeline for MENTOR-RL foregoes the use of proprietary judge models to ensure reproducibility and compliance with terms of use. Rewards are computed deterministically over structured state transitions and module-derived ground truth (MENTOR-derived and RWR-LOE) rather than by semantic evaluation from a frontier model. Trajectories are generated either by sampling the policy under training or by deterministic templating, not by proxy labeling through a closed API. One exception is query paraphrasing during dataset construction, for which we will use an open-source model with a pinned version so that the paraphrasing step can be reproduced without API access. Despite avoiding proprietary models during training, we will still compare MENTOR-RL to them through blind comparative evaluation, detailed in Section 0.2.11.

0.4.2 Limitations

Several limitations constrain the scope of the claims we can make with this work.

Reward-hacking risk. The composite module-membership score rewards moves toward the hidden target C . Since C is never observed directly, the agent cannot short-circuit it, but shortcut policies are still possible. For example, a policy that always returns a single-pass random-walk-with-restart expansion from the seed set will recover a substantial fraction of MENTOR modules without genuine multi-step reasoning. The efficiency penalty does not counteract this behavior. We will monitor tool-use entropy and trajectory diversity during training and, if shortcut behavior emerges, introduce adversarially constructed modules or a diversity bonus as mitigations.

Shared-weight verifier. The actor and verifier are two prompting modes of the same policy, so DPO and RL updates on one mode also update the other. This introduces a risk of verifier collapse, in which the verifier learns to endorse the actor’s proposals regardless of quality. We will monitor the verifier disagreement rate and, if collapse is observed, consider separate LoRA adapters per mode or a consistency regularizer between modes.

Task difficulty imbalance. The four task types are constructed uniformly during corpus construction, but they are not of equal difficulty; **explanation** tasks require no graph modification, while **refinement** tasks require the verifier to remove members

without a tool that directly marks membership. We treat the per-task sampling weights as a hyperparameter and expect to tune them during pilot training.

Split-level leakage. Dendrogram modules share genes across different cuts of the tree; therefore, a module-level split does not by itself prevent leakage through genes shared across overlapping modules. We will report overlap statistics between train and held-out splits and, if overlap is material, adopt a gene-overlap-aware split strategy.

Expert evaluation scope. The blind comparative evaluation relies on 5 to 15 domain experts making pairwise preference judgments. This sample size supports descriptive comparison but not tight confidence intervals on preference rates, and expert rating reliability will be reported alongside preference proportions.

0.4.3 Conclusion

We have presented MENTOR-RL, a proposed open-source reasoning agent for biological mechanistic interpretation, trained through a staged pipeline of supervised fine-tuning, direct preference optimization over shared-prefix trajectory branches, and policy gradient optimization for long-horizon exploration. The training environment grounds rewards in deterministic graph operators and module targets drawn from the MENTOR-derived and RWR-LOE corpora, removing proprietary judges from the loop and exposing every optimization signal as a reproducible, structured quantity. We view MENTOR-RL as one instance of a broader methodological pattern: domain-specialized scientific agents whose behavior is shaped by deterministic structured-state rewards over curated ground truth, rather than by semantic evaluation from frontier models. If successful, this pattern offers a route toward scientific AI systems that are cheap to deploy, auditable in their reasoning, and reproducible across research groups without centralized model access.

Bibliography

- [1] Shibli Abdulla et al. “CZ CELLxGENE Discover: a single-cell data platform for scalable exploration, analysis and modeling of aggregated data”. In: *Nucleic Acids Research* 53.D1 (2025), pp. D886–D900.
- [2] Redacted by arXiv. *The Llama 4 Herd: Architecture, Training, Evaluation, and Deployment Notes*. 2026. arXiv: [2601.11659 \[cs.SE\]](https://arxiv.org/abs/2601.11659). URL: <https://arxiv.org/abs/2601.11659>.
- [3] Scott Atchley et al. “Frontier: Exploring Exascale”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC ’23. Denver, CO, USA: Association for Computing Machinery, 2023. ISBN: 9798400701092. DOI: [10.1145/3581784.3607089](https://doi.org/10.1145/3581784.3607089). URL: <https://doi.org/10.1145/3581784.3607089>.
- [4] Albert-László Barabási, Natali Gulbahce, and Joseph Loscalzo. “Network medicine: a network-based approach to human disease”. In: *Nature reviews genetics* 12.1 (2011), pp. 56–68.
- [5] Albert-Laszlo Barabasi and Zoltan N Oltvai. “Network biology: understanding the cell’s functional organization”. In: *Nature reviews genetics* 5.2 (2004), pp. 101–113.
- [6] Sumanta Basu et al. “Iterative random forests to discover predictive and stable high-order interactions”. In: *Proceedings of the National Academy of Sciences* 115.8 (2018), pp. 1943–1948.
- [7] Lukas Biewald. *Experiment Tracking with Weights and Biases*. Software available from wandb.com. 2020. URL: <https://www.wandb.com/>.
- [8] Katy Börner et al. “Anatomical structures, cell types and biomarkers of the Human Reference Atlas”. In: *Nature Cell Biology* 23.11 (2021), pp. 1117–1128.
- [9] Yiqun Chen and James Zou. “Simple and effective embedding model for single-cell biology built from ChatGPT”. In: *Nature biomedical engineering* 9.4 (2025), pp. 483–493.
- [10] Paul F Christiano et al. “Deep reinforcement learning from human preferences”. In: *Advances in neural information processing systems* 30 (2017).

- [11] Ashley Cliff et al. “A high-performance computing implementation of iterative random forest for the creation of predictive expression networks”. In: *Genes* 10.12 (2019), p. 996.
- [12] Europe PMC Consortium. “Europe PMC: a full-text literature database for the life sciences and platform for innovation”. In: *Nucleic acids research* 43.D1 (2015), pp. D1042–D1048.
- [13] Lenore Cowen et al. “Network propagation: a universal amplifier of genetic associations”. In: *Nature Reviews Genetics* 18.9 (2017), pp. 551–562.
- [14] Manlio De Domenico et al. “Mathematical formulation of multilayer networks”. In: *Physical Review X* 3.4 (2013), p. 041022.
- [15] Alexander D Diehl et al. “The Cell Ontology 2016: enhanced content, modularization, and ontology interoperability”. In: *Journal of Biomedical Semantics* 7 (2016), p. 44.
- [16] Daya Guo et al. “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning”. In: *arXiv preprint arXiv:2501.12948* (2025).
- [17] Leland H Hartwell et al. “From molecular to modular cell biology”. In: *Nature* 402.Suppl 6761 (1999), pp. C47–C52.
- [18] Edward J Hu et al. “Lora: Low-rank adaptation of large language models.” In: *Iclr* 1.2 (2022), p. 3.
- [19] Trey Ideker and Nevan J Krogan. “Differential network biology”. In: *Molecular systems biology* 8 (2012), p. 565.
- [20] ISO. *ISO/IEC 14882:2011 Information technology — Programming languages — C++*. Geneva, Switzerland: International Organization for Standardization, Feb. 2012. URL: <http://www.iso.org>.
- [21] David Kainer et al. “RWRtoolkit: multi-omic network analysis using random walks on multiplex networks in any species”. In: *GigaScience* 14 (2025), giae028.
- [22] Sebastian Köhler et al. “Walking the interactome for prioritization of candidate disease genes”. In: *The American Journal of Human Genetics* 82.4 (2008), pp. 949–958.
- [23] Takeshi Kojima et al. “Large language models are zero-shot reasoners”. In: *Advances in neural information processing systems* 35 (2022), pp. 22199–22213.
- [24] Kenneth Li et al. “Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task”. In: *International Conference on Learning Representations (ICLR)*. 2023.
- [25] Pan Lu et al. “Eubiota: Modular Agentic AI for Autonomous Discovery in the Gut Microbiome”. In: *bioRxiv* (2026). DOI: [10.64898/2026.02.27.708412](https://doi.org/10.64898/2026.02.27.708412). eprint: <https://www.biorxiv.org/content/early/2026/02/28/2026.02.27.708412.full.pdf>. URL: <https://www.biorxiv.org/content/early/2026/02/28/2026.02.27.708412>.

- [26] Aman Madaan et al. “Self-refine: Iterative refinement with self-feedback”. In: *Advances in neural information processing systems* 36 (2023), pp. 46534–46594.
- [27] Jörg Menche et al. “Uncovering disease-disease relationships through the incomplete interactome”. In: *Science* 347.6224 (2015), p. 1257601.
- [28] Peter J Mucha et al. “Community structure in time-dependent, multiscale, and multiplex networks”. In: *science* 328.5980 (2010), pp. 876–878.
- [29] Christopher J Mungall et al. “Uberon, an integrative multi-species anatomy ontology”. In: *Genome Biology* 13.1 (2012), R5.
- [30] Reiichiro Nakano et al. “Webgpt: Browser-assisted question-answering with human feedback”. In: *arXiv preprint arXiv:2112.09332* (2021).
- [31] NVIDIA. *NVIDIA Nemotron 3: Efficient and Open Intelligence*. White Paper. 2025. URL: <https://arxiv.org/abs/2512.20856>.
- [32] OpenAI. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025. arXiv: [2508.10925](https://arxiv.org/abs/2508.10925) [cs.CL]. URL: <https://arxiv.org/abs/2508.10925>.
- [33] Long Ouyang et al. “Training language models to follow instructions with human feedback”. In: *Advances in neural information processing systems* 35 (2022), pp. 27730–27744.
- [34] Pranav Putta et al. “Agent q: Advanced reasoning and learning for autonomous ai agents”. In: *arXiv preprint arXiv:2408.07199* (2024).
- [35] Yujia Qin et al. “Toollm: Facilitating large language models to master 16000+ real-world apis”. In: *arXiv preprint arXiv:2307.16789* (2023).
- [36] Rafael Rafailov et al. “Direct preference optimization: Your language model is secretly a reward model”. In: *Advances in neural information processing systems* 36 (2023), pp. 53728–53741.
- [37] Jeff Rasley et al. “Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters”. In: *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2020, pp. 3505–3506.
- [38] G. van Rossum. *Python tutorial*. Tech. rep. CS-R9526. Amsterdam: Centrum voor Wiskunde en Informatica (CWI), May 1995.
- [39] Timo Schick et al. “Toolformer: Language models can teach themselves to use tools”. In: *Advances in neural information processing systems* 36 (2023), pp. 68539–68551.
- [40] John Schulman et al. “Proximal policy optimization algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).
- [41] Zhihong Shao et al. “Deepseekmath: Pushing the limits of mathematical reasoning in open language models”. In: *arXiv preprint arXiv:2402.03300* (2024).

- [42] Noah Shinn et al. “Reflexion: Language agents with verbal reinforcement learning”. In: *Advances in neural information processing systems* 36 (2023), pp. 8634–8652.
- [43] Aaditya Singh et al. *OpenAI GPT-5 System Card*. 2025. arXiv: [2601.03267](https://arxiv.org/abs/2601.03267) [cs.CL]. URL: <https://arxiv.org/abs/2601.03267>.
- [44] Kyle A. Sullivan et al. “MENTOR: Multiplex Embedding of Networks for Team-Based Omics Research”. In: *bioRxiv* (2024). DOI: [10.1101/2024.07.17.603821](https://doi.org/10.1101/2024.07.17.603821).
- [45] Shawn Zheng Kai Tan et al. “Brain Data Standards – A method for building data-driven cell-type ontologies”. In: *Scientific Data* 10 (2023), p. 50.
- [46] Kimi Team et al. *Kimi K2: Open Agentic Intelligence*. 2026. arXiv: [2507.20534](https://arxiv.org/abs/2507.20534) [cs.LG]. URL: <https://arxiv.org/abs/2507.20534>.
- [47] A. H. Vlot et al. “The MENTOR Interpretation Agent: From Network Embeddings to Mechanistic Narratives via Retrieval-Augmented LLMs”. In: *PASC 2025*. 2025.
- [48] Zhizheng Wang et al. “GeneAgent: self-verification language agent for gene-set analysis using domain databases”. In: *Nature Methods* 22.8 (2025), pp. 1677–1685.
- [49] Jason Wei et al. “Chain-of-thought prompting elicits reasoning in large language models”. In: *Advances in neural information processing systems* 35 (2022), pp. 24824–24837.
- [50] Leandro von Werra et al. *TRL: Transformers Reinforcement Learning*. <https://github.com/huggingface/trl>. 2020.
- [51] Chunlei Wu, Ian Macleod, and Andrew I Su. “BioGPS and MyGene.info: organizing online, gene-centric information”. en. In: *Nucleic Acids Res.* 41.Database issue (Jan. 2013), pp. D561–5.
- [52] Shunyu Yao et al. “React: Synergizing reasoning and acting in language models”. In: *The eleventh international conference on learning representations*. 2022.
- [53] Andy B. Yoo, Morris A. Jette, and Mark Grondona. “SLURM: Simple Linux Utility for Resource Management”. In: *Job Scheduling Strategies for Parallel Processing*. Springer Berlin Heidelberg, 2003, pp. 44–60. DOI: [10.1007/10968987_3](https://doi.org/10.1007/10968987_3).